

Chapter 4

Object-Oriented Programming (OOPS) Concepts

1. What Is OOPS (Object-Oriented Programming System)?

- OOPS stands for Object-Oriented Programming System.
- It is a programming approach where programs are designed using objects that represent real-world entities.
- Each object represents a real-world entity that has attributes (data/variables) and behaviors (methods/functions).

Example: Think of a Car

- Object: Car
- Attributes: color, brand, speed
- Behaviors: start(), stop(), accelerate()

2. Necessity of OOPS

- Before OOPS, programming was done in Procedural Languages (like C) — where data and functions were separate.
- OOPS combines both data and behavior into one unit (object), making programs more modular, reusable, and secure.

Why OOPS Is Needed

1. Real-world modeling (e.g., Car, Employee, Student).
2. Code reusability (use classes again and again).
3. Easy to maintain and update.
4. Data security using encapsulation.
5. Simplifies complex programs using modular structure.

3. Advantages of OOPS

Advantage	Description
1. Reusability	Code can be reused using classes and inheritance.
2. Data Hiding	Sensitive data can be hidden using access modifiers.
3. Modularity	Code is divided into objects — easier to manage and debug.
4. Extensibility	New features can be added easily without changing existing code.
5. Maintainability	Easier to update, test, and maintain.
6. Real-world Modeling	Represents real-world objects (e.g., Student, Employee).

4. OOPS Design with a Real-Time Example

Real-Life Analogy — imagine a School Management System:

- Classes: Student, Teacher, Subject, School
- Objects: Specific students, teachers, subjects
- Attributes: name, age, subject, marks
- Methods: attendClass(), teach(), calculateMarks()

5. The Six Core OOPs Concepts

1. Class
2. Object
3. Inheritance
4. Encapsulation
5. Polymorphism
6. Abstraction

6. What Is a Class?

A class is a blueprint or template for creating objects. It defines the data (attributes) and functions (methods) that the objects will have.

Real-world Example: A class is like an Architect's blueprint for building houses. The blueprint (class) itself doesn't occupy space — the houses (objects) built from it do.

7. What Is an Object?

An object is an instance of a class.

Object calling syntax: `classname objname = new classname();`

- It is a real-world entity created using the class blueprint.
- Each object has its own copy of the attributes defined in the class.
- When you create an object, memory is allocated to it.

8. How to Create a Class and Object in Java

```
class ClassName {  
    // data members  
    // methods  
}  
  
public class MainClass {  
    public static void main(String[] args) {  
        ClassName obj = new ClassName(); // object creation  
        obj.methodName();                // calling method  
    }  
}
```

9. Worked Examples — Classes, Objects, Attributes & Methods

9.1 Student Class

```
class Student {
    // Attributes (Data)
    String name;
    int age;
    String subject;

    // Method (Behavior)
    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Subject: " + subject);
    }
}

public class StudentMain {
    public static void main(String[] args) {
        // Object creation
        Student s1 = new Student();
        s1.name = "John";
        s1.age = 18;
        s1.subject = "Computer Science";
        s1.displayInfo();
    }
}
```

9.2 Car Class

```
class Car {
    String brand;
    int speed;

    void drive() {
        System.out.println("The car is driving");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c1 = new Car(); // object creation
        c1.brand = "Toyota";
        c1.speed = 120;
        c1.drive();
    }
}
```

```
}
```

9.3 Animal Class — Multiple Objects

```
class Animal {
    String name;
    void sound() {
        System.out.println(name + " makes a sound!");
    }
}

public class AnimalMain {
    public static void main(String[] args) {
        Animal dog = new Animal();
        dog.name = "Dog";
        dog.sound();

        Animal cat = new Animal();
        cat.name = "Cat";
        cat.sound();
    }
}
```

Dog makes a sound!

Cat makes a sound!

9.4 Simple Class and Object (Fixed Values)

```
class Student {
    String name = "Ravi";
    int age = 20;

    void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
    }
}

public class Example1 {
    public static void main(String[] args) {
        Student s = new Student(); // creating object
        s.displayInfo();           // calling method
    }
}
```

9.5 Creating Multiple Objects from One Class

```
class Car {
    String brand;
```

```

    int price;

    void show() {
        System.out.println("Brand: " + brand + ", Price: " + price);
    }
}

public class Example2 {
    public static void main(String[] args) {
        Car c1 = new Car();
        c1.brand = "Tata";
        c1.price = 800000;

        Car c2 = new Car();
        c2.brand = "Honda";
        c2.price = 1000000;

        c1.show();
        c2.show();
    }
}

```

9.6 Real-Life OOP Design — Bank Account

```

class BankAccount {
    String accountHolder;
    double balance;

    void deposit(double amount) {
        System.out.println("Deposited Amount: " + amount);
        balance += amount;
    }

    void display() {
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Balance: " + balance);
    }
}

public class Example3 {
    public static void main(String[] args) {
        BankAccount acc1 = new BankAccount();
        acc1.accountHolder = "Priya";
        acc1.balance = 100000;
        acc1.display();
        acc1.deposit(5000);
        acc1.display();
    }
}

```

```
}  
}
```

9.7 Necessity of OOP — Managing Multiple Students

```
class Student {  
    String name;  
    int marks;  
  
    void setDetails(String n, int m) {  
        name = n;  
        marks = m;  
    }  
  
    void display() {  
        System.out.println(name + " scored " + marks + " marks");  
    }  
}  
  
public class Example4 {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Student s2 = new Student();  
  
        s1.setDetails("Karthik", 85);  
        s2.setDetails("Renu", 92);  
  
        s1.display();  
        s2.display();  
    }  
}
```

This shows why OOP is useful: one class (Student) can produce as many independent objects as needed, each keeping its own separate name and marks — without writing repetitive code for each student.

10. Encapsulation

- Encapsulation is the concept of wrapping data (variables) and methods (functions) together into a single unit — a class — and restricting direct access to some of the object's data.
- It protects the internal state of an object by using access modifiers (private, public, protected).

Real-World Example — A Bank ATM

- You can't directly open the bank's database to change your balance.
- You use controlled interfaces (ATM options) to deposit or withdraw money.
- The internal process is hidden from you — that's encapsulation.

11. Abstraction

Abstraction means showing only essential features and hiding complex implementation details from the user.

In Java, abstraction is achieved using:

- Abstract classes (abstract keyword)
- Interfaces

Real-World Example — Driving a Car

- You only know how to use the steering wheel, accelerator, and brakes.
- You don't need to know how the engine converts fuel into motion — that's abstraction.

12. Inheritance

Inheritance allows one class (child or subclass) to acquire properties and behaviors (fields and methods) from another class (parent or superclass). It promotes code reusability.

Real-World Example

- A child inherits characteristics from their parents (like eye color, height).
- In programming, a Dog class inherits behavior from a general Animal class.

13. Polymorphism

Polymorphism means "many forms." It allows one interface to be used for different types of objects.

There are two types in Java:

1. Compile-time polymorphism (Method Overloading)
2. Runtime polymorphism (Method Overriding)

Real-World Example

A remote control works differently for a TV, AC, or Music System — same interface (the remote), different behaviors — that's polymorphism.

14. Java OOPs Concepts — Summary Table

Concept	Description	Java Feature / Syntax	Real-world Example
Class	A blueprint or template used to create objects. Defines attributes (variables) and behaviors (methods).	<pre>class Car { String color; void start() {...} }</pre>	A car design blueprint that defines structure and behavior.
Object	An instance of a class. Represents a real-world entity created from the class blueprint.	<pre>Car myCar = new Car();</pre>	An actual car built from the blueprint — e.g., your red Toyota.

Encapsulation	Hiding data and providing controlled access through methods (getters/setters).	private fields, public getters/setters	A bank account — you can deposit or withdraw, but not directly edit the balance.
---------------	--	--	--

(Inheritance, Polymorphism, and Abstraction follow the same pattern — a Java mechanism paired with a real-world analogy)

15. Tips for Learning and Applying OOP Concepts

1. Always connect a concept to a real-world analogy first. If you can't explain a class, object, or pillar using a real-life example (car, bank account, remote control), you likely don't fully understand it yet — go back to the analogy before writing code.
2. Remember the class vs. object distinction: a class defines what is possible (attributes + methods); an object is an actual instance using memory. A blueprint never IS a house — it only describes one.
3. Every object gets its own copy of the class's attributes. Changing `s1.name` never affects `s2.name` — this is exactly why OOP scales well for managing many similar entities (see the Student example with `s1` and `s2`).
4. Learn the 4 pillars in pairs, since they overlap conceptually: Encapsulation + Abstraction are both about hiding something (data vs. complexity); Inheritance + Polymorphism are both about reusing and adapting behavior across related classes.
5. When identifying encapsulation in code, look for private fields with public getter/setter methods, or restricted direct access — that access control is the actual mechanism, not just "grouping data together."
6. When identifying abstraction, ask "what is the user NOT shown?" — abstraction is about hiding implementation, while encapsulation is about hiding and protecting data. They're related but answer different questions.
7. For inheritance, always identify the parent (superclass) and child (subclass) first, and confirm the relationship reads naturally as "a [child] IS-A [parent]" (e.g., "a Dog IS-A Animal"). If that sentence doesn't make sense, inheritance is probably the wrong tool.
8. For polymorphism, distinguish the two types clearly: overloading = same method name, different parameters, decided at compile time; overriding = child class redefines a parent's method, decided at runtime.
9. Practice writing the class first, then a separate Main/Runner class to create objects and call methods — this mirrors real Java project structure and reinforces that classes and their usage are separate concerns.
10. Trace through object creation step by step for tricky examples: write down what memory is allocated, what values each object holds, and what each method call prints — exactly like the Dog/Cat or Student `s1/s2` examples above.